Developer Setup Guide

AI-Powered Career Preparation Application

This guide walks through everything needed to go from a blank machine to a running full-stack development environment for the capstone project described in the Software Design Document (SDD): a **Flutter** mobile app, a **Node.js/Express.js** REST API, **Firebase** (Auth, Firestore, Storage), and the **OpenAI API / Google Gemini API**.

Follow the sections in order — each one builds on the last.

---

0. Accounts You Will Need (create these first)

#	Account	Purpose	Cost
1	**Google Account**	Firebase project, Google Cloud Console	Free
2	**GitHub Account**	Source control, team collaboration	Free
3	**OpenAI Platform account** (platform.openai.com) *or* **Google AI Studio / Vertex AI** (for Gemini)	AI API access	Pay-as-you-go (both require billing setup; OpenAI needs a card on file, Gemini has a free tier)
4	**Apple Developer Account** (only if you will build/test on iOS or submit to the App Store)	iOS builds, TestFlight, App Store	US$99/year
5	**Google Play Console account** (only if you will publish to Play Store)	Android app distribution	US$25 one-time
6	**Hosting platform account** — Render, Railway, or Google Cloud (pick one)	Backend deployment	Free tier available

> Tip: One team member should create the Firebase project and the AI API account, then add the rest of the team as collaborators/editors so you're not juggling multiple billing accounts.

---

1. Core Developer Tools (install on every team member's machine)

1.1 Git

```
# Windows: download from git-scm.com
# macOS
brew install git
# Ubuntu/Debian
sudo apt update && sudo apt install git -y

git --version   # verify
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

1.2 Visual Studio Code (recommended IDE)

Download from https://code.visualstudio.com/. Install these extensions:

- **Flutter** and **Dart** (Dart-Code.flutter / Dart-Code.dart-code)
- **ESLint** and **Prettier** (for the Node.js backend)
- **Firebase** (toba.vsfire) — optional, syntax highlighting for security rules
- **Thunder Client** or use Postman (see §7) for API testing
- **GitLens** — optional, helps with Git history

1.3 Node.js (backend runtime)

Install the **LTS** version (v20.x as of this writing) via nvm so each teammate can pin the same version.

```
# macOS/Linux — install nvm first
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc   # or ~/.zshrc

nvm install --lts
nvm use --lts
node -v      # e.g. v20.x.x
```

```
npm -v

# Windows — download the LTS installer from nodejs.org, or use nvm-windows:
# https://github.com/coreybutler/nvm-windows
nvm install lts
nvm use lts
```

1.4 Flutter SDK (frontend)

```
# macOS
brew install --cask flutter
# or download the zip from flutter.dev and add flutter/bin to PATH

# Windows / Linux: download the SDK archive from
# https://docs.flutter.dev/get-started/install and extract it, then
# add <extracted-path>/flutter/bin to your PATH environment variable.

flutter --version
flutter doctor       # run this — it tells you exactly what's missing
```

Resolve every warning `flutter doctor` reports (Android toolchain, Xcode, VS Code plugin, etc.) before moving on — this single command is the fastest way to catch a broken setup early.

1.5 Android Studio (required even if you use VS Code, for the Android SDK/emulator)

1. Download from https://developer.android.com/studio
2. During setup, install: **Android SDK**, **Android SDK Platform-Tools**, **Android Virtual Device (AVD)**
3. Open Android Studio → **More Actions** → **SDK Manager** → confirm the latest stable Android SDK Platform is installed
4. Create an emulator: **More Actions** → **Virtual Device Manager** → **Create Device** (pick a Pixel profile + latest system image)
5. Accept Android licenses:

```
flutter doctor --android-licenses
```

1.6 Xcode (macOS only, required for iOS builds/testing)

```
# Install from the Mac App Store, then:
sudo xcode-select --switch /Applications/Xcode.app/Contents/Developer
sudo xcodebuild -runFirstLaunch
```

```
flutter doctor    # should show Xcode as ✓
```

Windows/Linux teammates cannot build for iOS locally — they should focus on Android during development, and have the one Mac-owning teammate handle iOS builds/TestFlight.

1.7 Docker Desktop (for containerized backend + local Firebase emulators)

Download from https://www.docker.com/products/docker-desktop/. Used later for deployment (Section 8) and optional local containerization.

---

2. Firebase Project Setup

2.1 Create the project

1. Go to https://console.firebase.google.com → **Add project**
2. Name it, e.g., `careerprep-ai` (use a separate project per environment later — see §2.7)
3. Disable Google Analytics for now (optional, can enable later)

2.2 Register your apps

In **Project Settings** → **Your apps**, register:

- An **Android app** (package name, e.g., `com.yourteam.careerprep`)
- An **iOS app** (bundle ID, e.g., `com.yourteam.careerprep`) — only if targeting iOS
- A **Web app** (optional, only needed if you build an admin panel)

2.3 Enable Authentication

Build → **Authentication** → **Get started** → **Sign-in method**, enable:

- **Email/Password**
- **Google** (fill in a support email when prompted)

2.4 Enable Cloud Firestore

Build → **Firestore Database** → **Create database**

- Start in **Production mode** (we'll write explicit security rules — never leave it in open test mode)

- Choose a region close to your primary users (e.g., `asia-southeast1` for the Philippines)

2.5 Enable Cloud Storage

Build → **Storage** → **Get started** → same region as Firestore.

2.6 Install the Firebase CLI and FlutterFire CLI

npm install -g firebase-tools
firebase login

dart pub global activate flutterfire_cli

2.7 (Recommended) Create three Firebase projects, not one

Project	Purpose
`careerprep-dev`	Local development, Firebase Emulator Suite
`careerprep-staging`	Sprint demos, adviser reviews
`careerprep-prod`	Final capstone deployment

This mirrors the environments described in SDD §11.5 and prevents test data or bugs from ever touching your "real" demo data. If timeline is tight, at minimum separate **dev** from **prod**.

2.8 Generate a service account key (for the backend)

Project Settings → **Service accounts** → **Generate new private key** → downloads a JSON file.

- Rename it `serviceAccountKey.json`
- **Never commit this file to Git** (add it to `.gitignore` immediately — see §4.2)
- Store it in your backend project's root (or better, load its contents from environment variables in production — see §4.5)

2.9 Draft initial Firestore Security Rules

Create `firestore.rules` (you'll deploy this from the Firebase CLI):

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Deny all direct client access by default —
    // all writes must go through the authenticated Express.js API
    // using the Admin SDK, which bypasses these rules.
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

Deploy it:

```
firebase init firestore   # select your project, accept default file names
firebase deploy --only firestore:rules
```

---

3. AI API Setup

Pick **one** provider to start (you can support both later via the `aiClient.js` wrapper described in SDD §4.3).

Option A — OpenAI API

1. Go to https://platform.openai.com → **Sign up**
2. **Settings** → **Billing** → add a payment method and set a monthly spend limit (important for a student project — cap it, e.g., at $10–20)
3. **Dashboard** → **API keys** → **Create new secret key** → copy it immediately (shown only once)
4. Note the model you'll target (e.g., `gpt-4o-mini` for cost efficiency during development)

Option B — Google Gemini API

1. Go to https://aistudio.google.com/ → sign in with your Google account
2. **Get API key** → **Create API key** (choose or create a Google Cloud project)
3. Gemini has a free tier suitable for development/testing — check current limits at https://ai.google.dev/pricing
4. Note the model you'll target (e.g., `gemini-1.5-flash` for cost efficiency)

Either way: treat this key exactly like a password. It will live only in your backend's `.env` file (§4.5) — never in the Flutter app, never in Git.

---

4. Backend Setup (Node.js + Express.js)

4.1 Scaffold the project

```
mkdir careerprep-backend && cd careerprep-backend
npm init -y
npm install express cors dotenv firebase-admin multer helmet
npm install express-rate-limit express-validator morgan
npm install openai            # if using OpenAI
npm install @google/generative-ai   # if using Gemini
npm install --save-dev nodemon jest supertest eslint prettier
```

4.2 Create `.gitignore` (do this before your first commit)

```
node_modules/
.env
serviceAccountKey.json
dist/
*.log
```

4.3 Recommended folder structure

```
careerprep-backend/
├── src/
│   ├── config/
│   │   ├── firebase.js        # Firebase Admin SDK init
│   │   └── aiClient.js        # OpenAI/Gemini wrapper
│   ├── middleware/
│   │   ├── authMiddleware.js  # verifies Firebase ID token
│   │   ├── errorHandler.js
│   │   └── validateRequest.js
│   ├── routes/
│   │   ├── authRoutes.js
│   │   ├── resumeRoutes.js
│   │   ├── interviewRoutes.js
│   │   ├── assessmentRoutes.js
│   │   └── dashboardRoutes.js
│   ├── services/
│   │   ├── resumeService.js
│   │   ├── interviewService.js
│   │   ├── assessmentService.js
```

```
│   │   └── recommendationService.js
│   ├── repositories/        # Firestore read/write functions
│   └── app.js
├── tests/
├── .env
├── .env.example
├── .gitignore
└── package.json
```

4.4 Initialize the Firebase Admin SDK

`src/config/firebase.js`:

```
const admin = require("firebase-admin");

admin.initializeApp({
  credential: admin.credential.cert({
    projectId: process.env.FIREBASE_PROJECT_ID,
    clientEmail: process.env.FIREBASE_CLIENT_EMAIL,
    privateKey: process.env.FIREBASE_PRIVATE_KEY.replace(/\\n/g, "\n"),
  }),
  storageBucket: process.env.FIREBASE_STORAGE_BUCKET,
});

module.exports = {
  auth: admin.auth(),
  db: admin.firestore(),
  storage: admin.storage(),
};
```

4.5 Create `.env` (copy values out of `serviceAccountKey.json`)

```
PORT=4000
NODE_ENV=development

FIREBASE_PROJECT_ID=careerprep-dev
FIREBASE_CLIENT_EMAIL=firebase-adminsdk-xxxxx@careerprep-dev.iam.gserviceaccount.com
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----\n"
FIREBASE_STORAGE_BUCKET=careerprep-dev.appspot.com

# pick whichever provider you're using
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxx
```

```
GEMINI_API_KEY=AIzaSyxxxxxxxxxxxxxxxxxxxxx

CORS_ORIGIN=http://localhost:3000
```

Also commit a `.env.example` with the same keys but blank/placeholder values, so teammates know what to fill in without ever seeing real secrets.

4.6 Minimal `app.js` to confirm everything boots

```
require("dotenv").config();
const express = require("express");
const cors = require("cors");
const helmet = require("helmet");
const morgan = require("morgan");

const app = express();
app.use(helmet());
app.use(cors({ origin: process.env.CORS_ORIGIN }));
app.use(express.json());
app.use(morgan("dev"));

app.get("/health", (req, res) => res.json({ status: "ok" }));

// app.use("/api/v1/auth", require("./src/routes/authRoutes"));
// app.use("/api/v1/resumes", require("./src/routes/resumeRoutes"));
// ...mount remaining routers as you build them

const PORT = process.env.PORT || 4000;
app.listen(PORT, () => console.log(`API running on http://localhost:${PORT}`));
```

4.7 Add convenience scripts to `package.json`

```
"scripts": {
  "dev": "nodemon src/app.js",
  "start": "node src/app.js",
  "test": "jest",
  "lint": "eslint ."
}
```

4.8 Run it

```
npm run dev
# visit http://localhost:4000/health -> {"status":"ok"}
```

---

5. Frontend Setup (Flutter)

5.1 Create the project

```
flutter create careerprep_app --org com.yourteam
cd careerprep_app
```

5.2 Connect Flutter to Firebase

```
flutterfire configure
```

This walks you through selecting your Firebase project and platforms (Android/iOS), and auto-generates `lib/firebase_options.dart`. Run it again any time you add a new platform or switch Firebase projects (e.g., dev → staging).

5.3 Add core dependencies

```
flutter pub add firebase_core firebase_auth cloud_firestore firebase_storage
flutter pub add google_sign_in
flutter pub add http dio                    # HTTP client for your Express API
flutter pub add provider flutter_riverpod     # pick one state-management approach
flutter pub add hive hive_flutter shared_preferences
flutter pub add file_picker                  # resume upload
flutter pub add speech_to_text               # mock interview voice input
flutter pub add fl_chart                     # dashboard charts
flutter pub add --dev flutter_lints build_runner
```

5.4 Initialize Firebase in `lib/main.dart`

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const CareerPrepApp());
}
```

5.5 Recommended folder structure

```
lib/
├── main.dart
├── firebase_options.dart
├── config/
│   └── api_config.dart        # backend base URL per environment
├── models/
├── services/                  # ApiService, AuthRepository, ResumeRepository...
├── providers/ (or riverpod/)
├── screens/
│   ├── auth/
│   ├── dashboard/
│   ├── resume/
│   ├── interview/
│   ├── assessment/
│   └── profile/
└── widgets/
```

5.6 Point the app at your local backend

`lib/config/api_config.dart`:

```
class ApiConfig {
  // Android emulator can't reach "localhost" directly — use 10.0.2.2
  static const String baseUrl = String.fromEnvironment(
    'API_BASE_URL',
    defaultValue: 'http://10.0.2.2:4000/api/v1',
  );
}
```

- **Android emulator** → backend on host machine: use `http://10.0.2.2:4000`
- **iOS simulator** → `http://localhost:4000` works fine
- **Physical device** → use your machine's LAN IP, e.g., `http://192.168.1.20:4000`, and make sure the device is on the same Wi-Fi network

5.7 Run the app

```
flutter devices            # confirm an emulator/device is available
flutter run
```

---

6. Local Development Workflow (running everything together)

Start Firebase Emulators (optional but recommended so you don't touch real cloud data while developing):

```
firebase init emulators   # select Auth, Firestore, Storagefirebase emulators:start
```

1. Point `src/config/firebase.js` at the emulator by setting `FIRESTORE_EMULATOR_HOST=localhost:8080` and `FIREBASE_AUTH_EMULATOR_HOST=localhost:9099` in `.env` during local dev.
2. **Start the backend**: `npm run dev` (in `careerprep-backend/`)
3. **Start the Flutter app**: `flutter run` (in `careerprep_app/`)
4. **Test endpoints directly** with Postman/Thunder Client before wiring up UI (see §7).

---

7. API Testing Setup

1. Install **Postman** (https://www.postman.com/downloads/) or use the **Thunder Client** VS Code extension.
2. Create a Postman **Collection** named "CareerPrep API" with one folder per module (Auth, Resume, Interview, Assessment, Dashboard) mirroring SDD §9.
3. Create a Postman **Environment** with variables: `base_url = http://localhost:4000/api/v1`, `id_token = <paste a real Firebase ID token here for testing>`.

To get a real ID token for manual testing without building the full login UI first, you can temporarily sign in through the Firebase Auth REST API:

```
curl -X POST \
"https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key=YOUR_WEB_API_KEY" \  -H "Content-Type: application/json" \  -d '{"email":"test@example.com","password":"Test1234!","returnSecureToken":true}'
```

4. Copy the returned `idToken` into your Postman environment variable and use it as `Authorization: Bearer {{id_token}}` on protected routes.

---

8. Version Control & Team Workflow

```
# One person creates the repos
git init
git add .
git commit -m "chore: initial project scaffold"
git branch -M main
git remote add origin https://github.com/yourteam/careerprep-backend.git
git push -u origin main
```

Repeat similarly for the Flutter app (either as a second repo, or both inside one monorepo with `/backend` and `/mobile` folders — a monorepo is simpler for a 4-person capstone team).

Suggested branch strategy:

- `main` — always deployable/demo-ready
- `develop` — integration branch for the current sprint
- `feature/<module-name>` — one branch per module (e.g., `feature/resume-analyzer`), merged into `develop` via Pull Request after review

Add a root `README.md` documenting how to run both projects, and protect `main` in GitHub (**Settings** → **Branches** → **Add rule** → require PR review before merge).

---

9. Deployment Setup (when ready to go beyond localhost)

9.1 Backend — containerize with Docker

`Dockerfile` in `careerprep-backend/`:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 4000
CMD ["node", "src/app.js"]
```

Test it locally:

```
docker build -t careerprep-backend .
```

docker run -p 4000:4000 --env-file .env careerprep-backend

9.2 Deploy the backend (pick one)

- **Render** (simplest): New → Web Service → connect your GitHub repo → set build command `npm install`, start command `npm start` → add all `.env` variables under **Environment**.
- **Railway**: `railway login` → `railway init` → `railway up`, then set environment variables in the dashboard.
- **Google Cloud Run**: `gcloud run deploy careerprep-backend --source . --region asia-southeast1 --allow-unauthenticated`, then set env vars via `gcloud run services update`.

After deployment, update `lib/config/api_config.dart` in Flutter to point at the live URL (e.g., `https://careerprep-backend.onrender.com/api/v1`) for staging/production builds, typically via `--dart-define=API_BASE_URL=...` when running `flutter build`.

9.3 Mobile app builds

Android release build
flutter build appbundle --dart-define=API_BASE_URL=https://your-api-url/api/v1

iOS release build (on macOS, with a valid signing certificate set up in Xcode)
flutter build ipa --dart-define=API_BASE_URL=https://your-api-url/api/v1

---

10. Consolidated Environment Variables Checklist

Variable	Where it lives	Example
`FIREBASE_PROJECT_ID`	backend `.env`	`careerprep-dev`
`FIREBASE_CLIENT_EMAIL`	backend `.env`	from service account JSON
`FIREBASE_PRIVATE_KEY`	backend `.env`	from service account JSON
`FIREBASE_STORAGE_BUCKET`	backend `.env`	`careerprep-dev.appspot.com`
`OPENAI_API_KEY` or `GEMINI_API_KEY`	backend `.env`	`sk-...` / `AIza...`

`CORS_ORIGIN`	backend `.env`	your Flutter web/dev origin, if any
`API_BASE_URL`	Flutter `--dart-define` at build time	`http://10.0.2.2:4000/api/v1`
`firebase_options.dart`	generated by `flutterfire configure`, safe to commit	—

Rule of thumb: anything that grants access to money (AI API keys) or admin-level data access (Firebase service account) stays server-side only and out of Git. Anything the mobile app needs (Firebase client config) is safe to ship inside the app, since it's protected by Firestore Security Rules and backend token verification, not secrecy.

---

11. Quick-Start Checklist

- [ ] All teammates have Node.js LTS, Flutter SDK, Git, Android Studio installed; `flutter doctor` shows no blockers
- [ ] Firebase project(s) created; Auth (Email/Password + Google) and Firestore and Storage enabled
- [ ] Firestore security rules deployed (deny direct client access)
- [ ] Service account key generated and added to `.gitignore`
- [ ] AI API key obtained (OpenAI or Gemini) with a spend limit set
- [ ] Backend repo scaffolded, `.env` populated, `npm run dev` returns `{"status":"ok"}` at `/health`
- [ ] Flutter project created, `flutterfire configure` run, app builds and launches on an emulator
- [ ] Postman collection created and able to hit `/health` and a sample authenticated route
- [ ] GitHub repo(s) created, `main`/`develop` branches set up, first commit pushed
- [ ] Team agrees on which AI provider (OpenAI vs. Gemini) to use first, and who owns the billing account

Once every box above is checked, you're ready to start Sprint 0 → Sprint 1 (User Authentication module) as outlined in SDD §2.4.

Development Environment Setup Guide

AI-Powered Career Preparation Application for Graduating College Students

This guide walks through everything your team needs to install, configure, and connect before writing a single line of feature code. It follows the stack defined in the Software Design Document: **Flutter** (frontend), **Node.js + Express.js** (backend), **Firebase** (Auth + Firestore + Storage), and the **OpenAI API / Google Gemini API** (AI features).

Work through the sections in order — each one depends on the one before it.

---

0. Prerequisites Checklist

Item	Needed For	Minimum Version
Windows 10/11, macOS 12+, or Ubuntu 20.04+	All development	—
At least 8 GB RAM (16 GB recommended)	Android emulator + IDE	—
~15 GB free disk space	SDKs, emulators, node_modules	—
Stable internet connection	Package downloads, Firebase, AI APIs	—
A Google account	Firebase Console access	—

A GitHub account	Version control / team collaboration	—
A credit/debit card or GCash (optional)	OpenAI billing (Gemini has a free tier)	—

Assign one team member as the **"environment owner"** who creates the shared Firebase project and API keys, then invites the rest of the team as collaborators — this avoids everyone accidentally creating separate, disconnected Firebase projects.

---

1. Install Git and Set Up the Repository

1. Install Git:
    - **Windows:** download from https://git-scm.com/download/win
    - **macOS:** `brew install git`
    - **Ubuntu:** `sudo apt update && sudo apt install git -y`

Configure your identity:
git config --global user.name "Your Name"git config --global user.email "your_email@example.com"

2.
3. Create the GitHub repository (one team member does this):
    - Create a new repo, e.g. `career-prep-ai-app`

Recommended structure:
career-prep-ai-app/ ├── mobile/ # Flutter app ├── backend/ # Node.js + Express API ├── docs/ # SDD, diagrams, meeting notes └── README.md

    -

Each member clones it:
git clone https://github.com/<org-or-user>/career-prep-ai-app.gitcd career-prep-ai-app

4.

Add a `.gitignore` at the root (critical — prevents committing secrets and build artifacts):
Nodebackend/node_modules/backend/.env# Fluttermobile/.dart_tool/mobile/build/mobile/.flutter-pluginsmobile/.flutter-plugins-dependenciesm

obile/android/key.propertiesmobile/ios/Flutter/Flutter.podspec#
Firebase**/google-services.json**/GoogleService-Info.plistserviceAccountKey.json#
OS.DS_Store

5.

⚠️ Firebase config files and service account keys should never be pushed to a public repo. If the repo is private and shared only among the team, some teams choose to commit `google-services.json` for convenience — but never commit `serviceAccountKey.json` or `.env` files containing API keys.

---

2. Set Up the Flutter Frontend Environment

2.1 Install Flutter SDK

1. Download Flutter from https://docs.flutter.dev/get-started/install (choose your OS).
2. Extract it to a permanent location (e.g., `C:\src\flutter` on Windows, `~/development/flutter` on macOS/Linux).
3. Add Flutter to your PATH:
    - **Windows:** Add `C:\src\flutter\bin` to Environment Variables → PATH.

macOS/Linux: add to `~/.zshrc` or `~/.bashrc`:
export PATH="$PATH:$HOME/development/flutter/bin"

    -

Verify installation:
flutter --versionflutter doctor

4. Resolve every ❌ or ⚠ that `flutter doctor` reports before continuing (it checks for Android toolchain, Xcode, IDE plugins, connected devices, etc.).

2.2 Install Android Studio (required even if you use VS Code)

1. Download from https://developer.android.com/studio
2. During setup, install the **Android SDK**, **Android SDK Command-line Tools**, and **Android Virtual Device (AVD)**.

Accept Android licenses:
flutter doctor --android-licenses

3.

4. Create a virtual device: Android Studio → **More Actions** → **Virtual Device Manager** → **Create Device** (choose a Pixel profile + a recent Android system image).

2.3 Install Xcode (macOS only, required for iOS builds)

1. Install Xcode from the Mac App Store.

Install command-line tools:
sudo xcode-select --switch /Applications/Xcode.app/Contents/Developersudo xcodebuild -runFirstLaunch

2.

Install CocoaPods (manages iOS dependencies):
sudo gem install cocoapods

3.

2.4 Set Up Your Code Editor

- **VS Code** (recommended for most of the team): install the **Flutter** and **Dart** extensions from the Extensions Marketplace.
- **Android Studio**: install the **Flutter** plugin (bundles the Dart plugin automatically) via **Settings** → **Plugins**.

2.5 Create the Flutter Project

cd career-prep-ai-app

flutter create mobile --org com.yourteam.careerprep

cd mobile

flutter run

Confirm the default counter app runs on an emulator or physical device before proceeding.

2.6 Add Core Frontend Packages

Edit `mobile/pubspec.yaml` and add (adjust versions as needed — run `flutter pub outdated` later to check for updates):

dependencies:

```
  flutter:
    sdk: flutter
  firebase_core: ^3.6.0
  firebase_auth: ^5.3.1
  cloud_firestore: ^5.4.4
  firebase_storage: ^12.3.4
  google_sign_in: ^6.2.1
  provider: ^6.1.2            # or flutter_riverpod
  http: ^1.2.2
  dio: ^5.7.0                 # optional, richer HTTP client
  hive: ^2.2.3
  hive_flutter: ^1.1.0
  shared_preferences: ^2.3.2
  file_picker: ^8.1.2         # resume upload
  speech_to_text: ^7.0.0      # mock interview voice input
  fl_chart: ^0.69.0           # dashboard charts
  google_fonts: ^6.2.1
```

Then run:

```
flutter pub get
```

---

3. Set Up the Node.js / Express.js Backend

3.1 Install Node.js

Install the **LTS** version from https://nodejs.org (or use a version manager):
 # Using nvm (recommended)curl -o-
https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bashnvm install --ltsnvm use
--lts

1.

Verify:
 node --versionnpm --version

2.

3.2 Scaffold the Backend Project

cd career-prep-ai-app

mkdir backend && cd backend

npm init -y

3.3 Install Core Dependencies

npm install express cors dotenv helmet morgan express-rate-limit express-validator multer

npm install firebase-admin

npm install openai # if using OpenAI API

npm install @google/generative-ai # if using Gemini API

npm install pdf-parse mammoth # resume text extraction (PDF / DOCX)

3.4 Install Development Dependencies

npm install --save-dev nodemon jest supertest eslint prettier

3.5 Recommended Backend Folder Structure

backend/

├── src/

```
│   ├── config/
│   │   ├── firebaseAdmin.js
│   │   └── aiClient.js
│   ├── middleware/
│   │   ├── authMiddleware.js
│   │   ├── errorHandler.js
│   │   └── validateRequest.js
│   ├── routes/
│   │   ├── authRoutes.js
│   │   ├── resumeRoutes.js
│   │   ├── interviewRoutes.js
│   │   ├── assessmentRoutes.js
│   │   └── dashboardRoutes.js
│   ├── services/
│   │   ├── resumeService.js
│   │   ├── interviewService.js
│   │   ├── assessmentService.js
│   │   └── recommendationService.js
│   ├── repositories/
│   │   └── userRepository.js
│   └── app.js
├── tests/
├── .env
├── .env.example
```

├── package.json

└── server.js

3.6 Add Convenience Scripts

In `backend/package.json`:

```
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js",
  "test": "jest --runInBand"
}
```

3.7 Minimal Server Bootstrap

`backend/server.js`:

```
require("dotenv").config();
const app = require("./src/app");
const PORT = process.env.PORT || 4000;
app.listen(PORT, () => console.log(`API running on port ${PORT}`));
```

`backend/src/app.js`:

```
const express = require("express");
const cors = require("cors");
const helmet = require("helmet");
const morgan = require("morgan");
```

```
const app = express();

app.use(helmet());

app.use(cors({ origin: process.env.ALLOWED_ORIGIN?.split(",") || "*" }));

app.use(express.json());

app.use(morgan("dev"));


app.get("/api/v1/health", (req, res) => res.json({ status: "ok" }));


// mount route files here, e.g.:

// app.use("/api/v1/auth", require("./routes/authRoutes"));


module.exports = app;
```

Run it:

```
npm run dev
```

Visit `http://localhost:4000/api/v1/health` — you should see `{"status":"ok"}`.

---

4. Set Up Firebase (Auth, Firestore, Storage)

4.1 Create the Firebase Project

1. Go to https://console.firebase.google.com
2. Click **Add project** → name it (e.g., `career-prep-ai-app`) → disable Google Analytics (optional, not required for this app) → **Create project**.

4.2 Register Your Apps

For the Flutter app (recommended: use FlutterFire CLI to automate this):

dart pub global activate flutterfire_cli

cd mobile

flutterfire configure

This walks you through selecting your Firebase project and automatically:

- Registers Android/iOS/web apps in Firebase
- Generates `lib/firebase_options.dart`
- Downloads `google-services.json` (Android) and `GoogleService-Info.plist` (iOS) into the correct folders

For the backend (Admin SDK):

1. In the Firebase Console: **Project Settings** → **Service Accounts** → **Generate new private key**.
2. Save the downloaded JSON as `backend/serviceAccountKey.json` (already in `.gitignore`).

Initialize the Admin SDK in `backend/src/config/firebaseAdmin.js`:
const admin = require("firebase-admin");const serviceAccount = require("../../serviceAccountKey.json");admin.initializeApp({ credential: admin.credential.cert(serviceAccount), storageBucket: "your-project-id.appspot.com",});module.exports = admin;

3. In production, prefer loading the service account from an environment variable rather than a committed file (see Section 6).

4.3 Enable Authentication Methods

1. Firebase Console → **Build** → **Authentication** → **Get started**.
2. Under **Sign-in method**, enable:
    - **Email/Password**
    - **Google** (set a support email when prompted)

In the Flutter app, initialize Firebase before `runApp()`:
void main() async { WidgetsFlutterBinding.ensureInitialized(); await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform); runApp(const MyApp());}

3.

4.4 Create Firestore Database

1. Firebase Console → **Build** → **Firestore Database** → **Create database**.
2. Choose **Production mode** (we'll write explicit security rules).
3. Select a location close to your users (e.g., `asia-southeast1`).
4. Create the collections listed in the SDD (`users`, `profiles`, `resumes`, `resume_feedback`, `interview_sessions`, `assessments`, `assessment_results`, `career_recommendations`, `readiness_scores`) — Firestore creates collections automatically the first time you write a document, so this can also happen naturally as you build features.

4.5 Set Firestore Security Rules

Since all writes should go through the authenticated Express API (using the Admin SDK, which bypasses security rules), lock down direct client access:

```
rules_version = '2';

service cloud.firestore {

  match /databases/{database}/documents {

    match /users/{uid} {

      allow read: if request.auth != null && request.auth.uid == uid;

      allow write: if false; // writes go through the backend only

    }

    match /{document=**} {

      allow read, write: if false;

    }

  }

}
```

Adjust as needed once you decide which reads (if any) the Flutter app will perform directly versus through the API.

4.6 Enable Cloud Storage

1. Firebase Console → **Build** → **Storage** → **Get started** → choose the same region as Firestore.
2. Set Storage rules similarly (deny direct client writes; uploads go through the backend, or use signed upload URLs).

4.7 Install the Firebase Emulator Suite (Local Development)

This lets your team test Auth/Firestore/Storage locally without touching production data or incurring cost.

npm install -g firebase-tools

firebase login

cd career-prep-ai-app

firebase init emulators

Select: Authentication, Firestore, Storage

firebase emulators:start

Point your local backend and Flutter app at the emulator ports (shown in the terminal, typically `localhost:9099` for Auth, `localhost:8080` for Firestore) during development.

---

5. Set Up the AI API (OpenAI or Gemini)

Pick one provider to start (the SDD's `aiClient.js` wrapper is designed so you can add the other later with minimal changes).

5.1 Option A — OpenAI API

1. Create an account at https://platform.openai.com
2. Go to **Dashboard** → **API keys** → **Create new secret key**. Copy it immediately (shown only once).
3. Go to **Billing** and add a payment method / prepaid credit — the API is pay-as-you-go.

Store the key in `backend/.env` (never in client code):
OPENAI_API_KEY=sk-...

4.

Quick test:
node -e "const OpenAI = require('openai');const client = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });client.chat.completions.create({ model: 'gpt-4o-mini', messages: [{ role: 'user', content: 'Say hello in one sentence.' }]}).then(r => console.log(r.choices[0].message.content));"

5.

5.2 Option B — Google Gemini API

1. Go to https://aistudio.google.com/app/apikey
2. Sign in with your Google account and click **Create API key**. Gemini offers a generous free tier, good for capstone development and testing.

Store it in `backend/.env`:
GEMINI_API_KEY=AIza...

3.

Quick test:
node -e "const { GoogleGenerativeAI } = require('@google/generative-ai');const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);const model = genAI.getGenerativeModel({ model: 'gemini-1.5-flash' });model.generateContent('Say hello in one sentence.') .then(r => console.log(r.response.text()));"

4.

5.3 Wrap the Provider Behind One Interface

`backend/src/config/aiClient.js` (sketch):

```
const provider = process.env.AI_PROVIDER || "openai"; // "openai" | "gemini"



async function generateText(prompt) {

  if (provider === "openai") {

    // call OpenAI here

  } else {

    // call Gemini here
```

```
  }
}

module.exports = { generateText };
```

This matches the "Integration Layer" design in the SDD (Section 4.3) and means switching providers later is a one-line `.env` change, not a rewrite.

---

6. Environment Variables & Secrets Management

Create `backend/.env` (copy from a committed `backend/.env.example` so teammates know what's required):

```
PORT=4000
ALLOWED_ORIGIN=http://localhost:3000
AI_PROVIDER=gemini
OPENAI_API_KEY=
GEMINI_API_KEY=
FIREBASE_PROJECT_ID=your-project-id
FIREBASE_STORAGE_BUCKET=your-project-id.appspot.com
```

`backend/.env.example` (safe to commit — no real values):

```
PORT=4000
ALLOWED_ORIGIN=
AI_PROVIDER=
OPENAI_API_KEY=
GEMINI_API_KEY=
```

FIREBASE_PROJECT_ID=

FIREBASE_STORAGE_BUCKET=

Share real secret values with teammates through a private channel (e.g., a password manager or your adviser-approved shared drive) — never through group chat screenshots or public commits.

---

7. API Testing Tools

1. Install **Postman** (https://www.postman.com/downloads) or **Insomnia**.
2. Create a collection called `Career Prep API` with folders matching the SDD's API groups: Auth, Resume, Interview, Assessment, Dashboard.
3. Set up a Postman **environment** with variables `{{baseUrl}}` (`http://localhost:4000/api/v1`) and `{{idToken}}` so you can paste a fresh Firebase ID token after logging in during testing.
4. To get a test ID token without building the full login UI yet, you can temporarily use the Firebase Auth REST API or a small test script with the Firebase client SDK.

---

8. Docker (for Later Deployment)

Install Docker Desktop (https://www.docker.com/products/docker-desktop) so the backend runs identically on every machine and on the hosting platform.

`backend/Dockerfile`:

FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm ci --omit=dev

COPY . .

EXPOSE 4000

CMD ["node", "server.js"]

Build and run locally to confirm it works before deploying:

docker build -t career-prep-api .

docker run -p 4000:4000 --env-file .env career-prep-api

---

9. Deployment Accounts (Set Up Once, Use in Sprint 6–7)

Service	Purpose	Setup Link
Render / Railway / Google Cloud Run	Hosting the containerized Express API	render.com / railway.app / cloud.google.com/run
Firebase Hosting (optional)	Hosting an admin web panel, if built	Included in Firebase Console
Google Play Console ($25 one-time)	Publishing the Android app	play.google.com/console
Apple Developer Program ($99/year)	Publishing the iOS app / TestFlight	developer.apple.com

For capstone defense purposes, Render/Railway's free or low-cost tiers plus a debug/ad-hoc mobile build are typically sufficient — you don't need to pay for app store accounts unless you intend to publish publicly.

---

10. Verification Checklist

Before starting Sprint 1 feature work, confirm every box below:

- [ ] `flutter doctor` shows no unresolved issues
- [ ] `flutter run` launches the starter app on an emulator/device
- [ ] `npm run dev` starts the Express server and `/api/v1/health` responds
- [ ] Firebase project created; Authentication (Email/Password + Google) enabled
- [ ] Firestore database created with security rules deployed
- [ ] Cloud Storage bucket created with security rules deployed
- [ ] `firebase_options.dart`, `google-services.json`, `GoogleService-Info.plist` present in the Flutter project
- [ ] `serviceAccountKey.json` present in the backend (and in `.gitignore`)
- [ ] AI provider API key obtained and test call returns a response
- [ ] `.env` and `.env.example` created; real secrets shared privately, not committed
- [ ] Firebase Emulator Suite runs locally for offline-safe development
- [ ] Postman collection created with a working authenticated test request
- [ ] Repository pushed to GitHub with `.gitignore` protecting all secrets
- [ ] Every team member has successfully cloned the repo and run both the Flutter app and the backend locally

Once every item is checked, the team is ready to begin Sprint 1 (User Authentication and User Profile modules) as outlined in Section 2.4 of the SDD.